

A Benchmark for Sinhala-Script Language Identification: Disambiguating Sinhala, Pali, and Sanskrit in Closed and Open World Contexts

Anonymous ACL submission

Abstract

Sinhala, Pali, and Sanskrit can all be written in Sinhala script. Distinguishing these languages remains challenging for existing language identification tools. We present a benchmark for these three languages and evaluate models in two settings. The first distinguishes only these languages. The second includes them alongside other languages. We compare seven baseline architectures using full sentences and shorter text fragments. Full sentences are classified accurately, but performance declines on single words. Character n -gram and subword models outperform the evaluated deep neural models on these single-word inputs. We also find that naive fine-tuning reduces performance on other evaluated languages. A two-stage routing pipeline improves identification of the target languages while helping preserve performance on the other languages. These findings highlight the importance of evaluating both short inputs and performance beyond the target languages.

1 Introduction

Language Identification (LangID) is a standard workhorse in modern NLP pipelines, quietly filtering web crawls, routing machine translation requests, and sorting text archives (Jauhainen et al., 2019; Lui and Baldwin, 2012). For widely used writing systems like Latin or Cyrillic, models are naturally trained to distinguish among dozens of languages sharing the same letters. But when a writing system is strongly associated with a single modern national language, models often make a coarse assumption: they treat the script itself as an orthographic monolith.

That assumption breaks down completely for the Sinhala script (Unicode block U+0D80–U+0DFF). In Sri Lanka and Theravada Buddhist scholarship, the Sinhala script has not been used solely for **Sinhala** (an Indo-Aryan language spoken by over 17 million people). For more than two millennia, it

has also been the primary written medium for **Pali** (the liturgical language of Theravada Buddhism) and **Sanskrit** (the classical scholarly language of South Asian philosophy, grammar, and medicine). In fact, the Theravada Buddhist canon (*Tipitaka*) was first committed to writing in the Sinhala script in the first century BCE at the Aluvihara rock temple in Matale, Sri Lanka (Walshe, 1983). Over the centuries, hundreds of thousands of palm-leaf manuscripts and modern printed editions have preserved Pali and Sanskrit entirely in Sinhala letters.

Because planetary-scale LangID tools—like fast-Text LID-176 (Joulin et al., 2017), GlotLID v3 (Kargaran et al., 2023), NLLB LID-218 (Costa-Jussà et al., 2022), and OpenLID-v2 (Burchell et al., 2023)—map every Sinhala character directly to modern Sinhala (sin or sin_Sinh), they have exactly zero recall on Sinhala-script Pali and Sanskrit. They lump all three languages into a single label, quietly corrupting multilingual corpora and web crawls.

In this paper, we examine this script-sharing challenge by addressing two central research questions:

1. **When context is stripped down to isolated words, what architectural inductive biases allow models to distinguish closely related languages written in the same script?**
2. **In an open-world setting with global background languages, how can foundation models be adapted to resolve local script confusion without destroying their pre-trained global knowledge?**

To answer the first question, we assemble a curated, document-isolated target corpus of 74,318 sentences covering Sinhala, Pali, and Sanskrit in the Sinhala script. We then evaluate seven from-scratch baseline architectures under severe context deprivation, showing why character n -grams and

subwords beat deep neural networks when inputs are cut down to single words.

To answer the second question, we evaluate foundation models on three 11-language hybrid benchmarks constructed from FLORES+, CommonLID, and WiLI-2018. We show that while naive fine-tuning induces severe catastrophic forgetting on global languages, a simple two-stage routing pipeline fixes script ambiguity with 0.0% degradation on background languages.

2 Dataset Construction and Taxonomy

2.1 Curated Target Corpus

Distinguishing languages within the same script requires clean, balanced data. We assembled a 74,318-sentence target corpus from historical, canonical, and digital sources (summarized in Table 1; full provenance in Appendix B):

- **Sinhala (Sinh-Sinh, 32,263 sentences):** Sourced from the SiPaKosa repository (historical and Buddhist biographical writing) and the Sinhala side of a sentence-aligned Pali-Sinhala parallel translation corpus.
- **Pali (Pali-Sinh, 29,317 sentences):** Drawn from the Pali side of the canonical translation corpus and the SiDiaC-v2.0 corpus (Jayatilleke and de Silva, 2025), containing Theravada *Tipitaka* texts, commentaries (*Atthakatha*), and grammatical treatises (Walshe, 1983).
- **Sanskrit (San-Sinh, 12,738 sentences):** Digitized Sanskrit in Sinhala script is genuinely hard to find. We combined three sources: **SansinNT** (7,953 sentences from a historical 19th-century Sanskrit New Testament printed in Sinhala script); classical poetry from the **Digital Corpus of Sanskrit** (DCS; 4,684 sentences) (Hellwig, 2010), transliterated from SLP1 Romanization into Sinhala script using the verified phonetic mappings of *Aksharamukha* (Rajan, 2023); and a devotional *kavaca* text (101 sentences) from SiDiaC-v2.0 (Jayatilleke and de Silva, 2025).

A common pitfall in LangID benchmarking is randomly shuffling sentences across splits. When sentences from the same book appear in both train and test sets, models easily score high simply by memorizing topical vocabulary. To prevent this, we enforced a strict **group-stratified split**: sentences

Table 1: Target Corpus splits and sentence counts ($N = 74,318$). Partitioned with strict document-level grouping.

Language Class	Train	Val	Test	Total
Sinhala (Sinh-Sinh)	26,675	2,895	2,693	32,263
Pali (Pali-Sinh)	23,490	2,800	3,027	29,317
Sanskrit (San-Sinh)	10,120	1,291	1,327	12,738
Total	60,285	6,986	7,047	74,318

from any given literary work or chapter were kept together as an atomic block and placed entirely into training (81.1%), validation (9.4%), or testing (9.5%).

2.2 11-Language BCP-47 Hybrid Taxonomy

In real applications, Sinhala-script text does not exist in a vacuum; it appears alongside regional and global languages. To test models in realistic conditions without leaking evaluation data, our taxonomy adopts the **BCP-47 standard** (Phillips and Davis, 2009) ([Language]-[Script]):

- **Target Classes:** Sinh-Sinh, Pali-Sinh, San-Sinh.
- **Global Background Classes:** San-Deva (Devanagari Sanskrit), Eng-Latn (English), Tam-Tam1 (Tamil), Hin-Deva (Hindi), Ben-Beng (Bengali), Ara-Arab (Arabic), Fre-Latn (French), and Ger-Latn (German).

We built three hybrid evaluation benchmarks by taking the official test partitions of FLORES+ (Goyal et al., 2022), CommonLID (Suarez et al., 2026), and WiLI-2018 (Thoma, 2018), replacing their monolithic Sinhala rows with our 7,047 held-out target sentences, and keeping Devanagari Sanskrit alongside background languages.

To train from-scratch baselines on this open-world task under clean conditions, we also constructed a balanced multi-language corpus of **110,000 sentences** (10,000 sentences per class across all 11 languages; 9,000 train / 1,000 val). Target sentences were drawn strictly from our target training split, ensuring zero overlap with the test set.

3 Analysis and Findings

3.1 Closed-World Baselines & The Fragment Test

We trained seven representative architectures strictly from scratch on our target training set:

Table 2: Closed-World Macro-F1 across four input lengths ($N = 7,047$). Full sentences are easy for all models, but single words expose stark architectural differences.

Architecture	Full Sent.	5-Word	3-Word	1-Word
Multinomial NB	0.9947	0.9782	0.9520	0.8584
fastText (Scratch)	0.9958	0.9768	0.9416	0.8027
Char-CNN	0.9950	0.9749	0.9388	0.7654
Char-BiGRU	0.9948	0.9710	0.9255	0.6916
Linear SVM	0.9960	0.9801	0.9532	0.6099
Logistic Regression	0.9955	0.9780	0.9482	0.6054
XGBoost	0.9842	0.9554	0.9102	0.5891

1. **Multinomial Naive Bayes (MNB):** Character n -grams ($n \in [1, 5]$) with sublinear TF-IDF scaling (McCallum and Nigam, 1998).
2. **Linear SVM:** One-vs-Rest formulation with $C = 1.0$ over character n -grams (Joachims, 1998).
3. **Logistic Regression (LogReg):** L2-regularized multinomial loss.
4. **XGBoost:** Gradient-boosted trees (100 estimators, max depth 6) (Chen and Guestrin, 2016).
5. **Char-CNN:** Three parallel 1D convolution banks (kernel sizes 3, 5, 7; 128 filters each) with global max-pooling (Zhang et al., 2015).
6. **Char-BiGRU:** Bidirectional GRU (128 units per direction) over 64-dimensional character embeddings.
7. **fastText (From Scratch):** Two-layer shallow network with 100-dimensional embeddings and subwords ($n \in [3, 6]$) (Joulin et al., 2017).

Full Sentences are Deceptively Easy. As Table 2 shows, when models have access to full-length sentences, every single architecture performs well. All seven models score above 0.984 Macro-F1, with Linear SVM leading at 0.9960. If one evaluates only on complete sentences, it appears that any standard classifier has solved the problem.

Real-world text, however, is rarely that neat. Search queries, dictionary headwords, and OCR output from damaged palm leaves often consist of single words or brief phrases. To see what happens under extreme contextual deprivation, we generated synthetic fragments by taking the first 5, 3, and 1 words of each test sentence.

As input length drops, performance splits wide open. On single words, the deep neural networks and margin classifiers fall apart: Linear SVM drops from 0.9960 down to 0.6099 (a drop of nearly 40 points), while Char-BiGRU sinks from 0.9948 to 0.6916.

Meanwhile, Multinomial Naive Bayes holds up with a Macro-F1 of **0.8584**, and fastText stays strong at **0.8027**.

Why Subwords and N -Grams Win on Short Text. This difference comes down to the inductive bias of each architecture. Recurrent networks and margin classifiers look for broader sentence structure and sequence flow. When reduced to an isolated word, they starve for context.

Sinhala, Pali, and Sanskrit, however, are highly inflected languages. Even in an isolated word, morphological clues are everywhere:

- **Pali** relies on distinctive nominal case endings (like the genitive singular in *-assa* or instrumental plural in *-ēhi*) and verbal endings (*-ti*, *-ntu*).
- **Sanskrit** features characteristic consonant clusters and case inflections (like *-āt*, *-bhyām*, and visarga forms).
- **Sinhala** uses modern colloquial postpositions and verbal markers (like *-vē*, *-ēya*, *-da*).

Because MNB calculates likelihoods over all overlapping character n -grams, it catches these diagnostic inflectional fragments directly. fastText does something similar through its subword bag-of-characters (Joulin et al., 2017). For short-text and OCR applications where sentence syntax is missing, simple subword and n -gram models are far more dependable than heavier neural architectures.

3.2 Open-World Benchmarks & The Forgetting Trap

In the real world, target texts are mixed with global background languages. How do existing foundation models and fine-tuned pipelines handle this?

Zero-Shot Collapse. As Table 3 shows, all off-the-shelf foundation models hover around 0.68–0.71 Macro-F1 across FLORES+, CommonLID, and WiLI-2018. The reason is straightforward: because every stock model assigns Sinhala script strictly to modern Sinhala, their recall on Sinhala-script Pali and Sanskrit is exactly 0.0%.

Table 3: Open-World 11-Language Macro-F1 across FLORES+, CommonLID, and WiLI-2018. Two-stage routing prevents catastrophic forgetting while resolving target ambiguity.

Model / Pipeline	FLORES+	CommonLID	WiLI-2018
<i>From-Scratch Baselines (110k Multi-Language Training)</i>			
fastText (From Scratch)	0.9634	0.9412	0.9580
Multinomial NB	0.9589	0.9284	0.9512
Linear SVM	0.9712	0.9488	0.9654
<i>Stock Foundation Models (Zero-Shot Baseline)</i>			
fastText LID-176	0.7021	0.6980	0.7015
OpenLID-v2	0.6894	0.6841	0.6880
NLLB LID-218	0.7042	0.7010	0.7033
GlottLID v3	0.7088	0.7025	0.7072
ConLID	0.7104	0.7052	0.7094
<i>Naive Fine-Tuning (Target Data Only)</i>			
fastText Fine-Tuned	0.2612	0.2584	0.2605
OpenLID Fine-Tuned	0.2590	0.2541	0.2578
<i>Two-Stage Specialist Routing (Proposed)</i>			
fastText LID-176 2-Stage	0.9721	0.9510	0.9702
OpenLID-v2 2-Stage	0.9685	0.9482	0.9664
NLLB LID-218 2-Stage	0.9734	0.9525	0.9718
GlottLID v3 2-Stage	0.9740	0.9540	0.9725
ConLID 2-Stage	0.9752	0.9561	0.9738

The Failure of Naive Fine-Tuning. A common knee-jerk reaction to classification errors is to take a pre-trained model and fine-tune it on target data. But when one fine-tunes fastText LID-176 or OpenLID-v2 exclusively on our three target languages, catastrophic forgetting kicks in immediately (McCloskey and Cohen, 1989; French, 1999). The model learns the three target classes, but its representations for other languages degrade rapidly; it begins predicting English, French, and Tamil as Sinhala or Pali. Across the 11-language benchmarks, Macro-F1 collapses to ≈ 0.26 .

Why Two-Stage Routing Just Works. Instead of fighting catastrophic forgetting with complex regularization or replay buffers, one can sidestep it entirely with a simple two-stage routing setup:

- **Stage 1 (General Router):** The frozen stock foundation model classifies the input text. If it predicts any non-Sinhala language (English, Tamil, Devanagari Sanskrit, etc.), that prediction is returned directly.
- **Stage 2 (Specialist):** If and only if Stage 1 predicts Sinhala script (si or sin_Sinh), the text is handed over to our fine-tuned specialist classifier to make the 3-way call between Sinhala, Pali, and Sanskrit.

Because the Stage 1 model is never modified, there

is **zero catastrophic forgetting** on background languages. At the same time, Stage 2 resolves the localized script confusion.

As Table 3 shows, Two-Stage Routing lifts every foundation model to 0.95–0.97 Macro-F1, beating both the zero-shot foundation models and the from-scratch multi-language baselines. ConLID 2-Stage achieves the highest overall scores (0.9752 on FLORES+, 0.9561 on CommonLID, 0.9738 on WiLI-2018).

3.3 The Dialect Continuum Headache

A final practical lesson comes from models with hundreds of fine-grained regional dialect heads, like OpenLID-v2. When evaluated on Devanagari Sanskrit in CommonLID, OpenLID-v2 scores an F1 of just 0.0276 (detailed in Appendix Table 5). Rather than predicting Sanskrit, it scatters predictions across closely related North Indian regional dialects (Awadhi, Bhojpuri, Maithili, and Hindi).

While adding fine-grained dialect labels is helpful for living spoken varieties, it creates serious confusion for classical languages that share ancient roots and vocabulary with those dialects. Models with cleaner, standardized language taxonomies (like GlottLID v3 and ConLID) are much more robust against this kind of fragmentation.

4 Conclusion

Treating the Sinhala script as a language label erases two thousand years of canonical Pali and classical Sanskrit literature from digital NLP pipelines. In this work, we built the first dedicated benchmark to study this problem. Under severe context deprivation (down to single words), character n -gram models and subword embeddings prove much more reliable than deep sequence models by picking up on inflectional affixes. In open-world settings, naive fine-tuning wipes out global language knowledge, but a modular two-stage routing pipeline eliminates script ambiguity while keeping pre-trained capabilities completely intact.

Our datasets, benchmark splits, and models are available at: <https://github.com/anonymous-submission/sinhala-script-langid>.

Limitations

We note three main limitations: First, because naively digitized Sanskrit in Sinhala script is scarce,

about 37% of our Sanskrit target data (4,684 sentences from the Digital Corpus of Sanskrit) was transliterated from SLP1 Romanization using *Aksharamukha*. While phonetically sound, digital transliteration does not capture all the historical scribal ligatures found in physical manuscripts. Second, our benchmark tests clean digital text. Severely degraded palm-leaf manuscripts with OCR noise, broken characters, and non-standard orthography require dedicated image preprocessing before LangID can be applied. Finally, while our 11-language hybrid benchmarks provide balanced testing across major language families, they do not cover all 200+ world languages simultaneously.

Ethics Statement

This research respects the cultural and religious importance of canonical Theravada texts (*Tipitaka*) and classical Sanskrit works. All data sources used in this work (SiPaKosa, SiDiaC-v2.0, SansinNT, DCS, FLORES+, CommonLID, WiLI-2018) are open-access, public-domain, or released under permissive research licenses. We do not redistribute copyrighted or private data. Our aim is to ensure that historical and low-resource South Asian languages are accurately represented in global language technologies.

References

Laurie Burchell, Alexandra Birch, Nikolay Bogoychev, and Kenneth Heafield. 2023. [An open dataset and model for language identification](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 865–879, Toronto, Canada. Association for Computational Linguistics.

Tianqi Chen and Carlos Guestrin. 2016. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pages 785–794.

Marta R Costa-Jussà, James Cross, Onur Çelebi, Maha Elbayad, Kenneth Heafield, Kevin Heffernan, Elahe Kalbassi, Janice Lam, Daniel Licht, Jean Maillard, et al. 2022. No language left behind: Scaling human-centered machine translation. *arXiv preprint arXiv:2207.04672*.

Robert M French. 1999. Catastrophic forgetting in connectionist networks. *Trends in cognitive sciences*, 3(4):128–135.

Naman Goyal, Cynthia Gao, Vishrav Chaudhary, Peng-Jen Chen, Guillaume Wenzek, Da Ju, Sanjana Krishnan, Marc’Aurelio Ranzato, Francisco Guzmán,

and Angela Fan. 2022. [The Flores-101 evaluation benchmark for low-resource and multilingual machine translation](#). *Transactions of the Association for Computational Linguistics*, 10:522–538.

Oliver Hellwig. 2010. DCS - the digital corpus of sanskrit. In *Proceedings of the 17th World Sanskrit Conference*. <https://github.com/ambuda-org/dcs>.

Tommi Jauhiainen, Marco Lui, Marcos Zampieri, Timothy Baldwin, and Krister Lindén. 2019. [Automatic language identification in texts: A survey](#). *Journal of Artificial Intelligence Research*, 65.

Nevidu Jayatilleke and Nisansa de Silva. 2025. [SiDiaC: Sinhala diachronic corpus](#). In *Proceedings of the 39th Pacific Asia Conference on Language, Information and Computation*, pages 511–527, Hanoi, Vietnam. Association for Computational Linguistics.

Thorsten Joachims. 1998. [Text categorization with support vector machines](#). *Proc. European Conf. Machine Learning (ECML’98)*.

Armand Joulin, Edouard Grave, Piotr Bojanowski, and Tomas Mikolov. 2017. [Bag of tricks for efficient text classification](#). In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers*, pages 427–431, Valencia, Spain. Association for Computational Linguistics.

Amir Hossein Kargaran, Ayyoob Imani, François Yvon, and Hinrich Schuetze. 2023. [GlotLID: Language identification for low-resource languages](#). In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 6155–6218, Singapore. Association for Computational Linguistics.

Marco Lui and Timothy Baldwin. 2012. [langid.py: An off-the-shelf language identification tool](#). In *Proceedings of the ACL 2012 System Demonstrations*, pages 25–30, Jeju Island, Korea. Association for Computational Linguistics.

Andrew McCallum and Kamal Nigam. 1998. A comparison of event models for naive bayes text classification. In *AAAI-98 Workshop on Learning for Text Categorization*, volume 752, pages 41–48.

Michael McCloskey and Neal J Cohen. 1989. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of learning and motivation*, volume 24, pages 109–165. Elsevier.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. 2011. Scikit-learn: Machine learning in python. *the Journal of machine Learning research*, 12:2825–2830.

Addison Phillips and Mark Davis. 2009. [Tags for identifying languages](#). RFC 5646, Internet Engineering Task Force.

Vinodh Rajan. 2023. Aksharamukha: Asian script converter. <https://aksharamukha.appspot.com/>.

Pedro Ortiz Suarez, Laurie Burchell, Catherine Arnett, Rafael Mosquera, Sara Hincapié Monsalve, Thom Vaughan, Damian Stewart, Malte Ostendorff, Idris Abdulmumin, Vukosi Marivate, Shamsuddeen Hassan Muhammad, Atnafu Lambebo Tonja, Hend Al-Khalifa, Nadia Ghezaiel Hammouda, Verrah Akinyi Otiende, Tack Hwa Wong, Jakhongir Saydaliev, Melika Nobakhtian, Muhammad Ravi Shulthan Habibi, and 78 others. 2026. [CommonLID: Re-evaluating state-of-the-art language identification performance on web data](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 33063–33080, San Diego, California, United States. Association for Computational Linguistics.

Martin Thoma. 2018. The wili benchmark dataset for written language identification. *arXiv preprint arXiv:1801.07779*.

Maurice Walshe. 1983. [Pali literature, including the canonical literature in prakrit and sanskrit of all the hinayana schools](#). k. r. norman. *Buddhist Studies Review*, 1(2):172–173.

Xiang Zhang, Junbo Zhao, and Yann LeCun. 2015. Character-level convolutional networks for text classification. *Advances in neural information processing systems*, 28.

A Detailed Hybrid Benchmark Results Across 11 Languages

Tables 4, 5, and 6 give the full per-class evaluation across all 11 target and background languages on FLORES+, CommonLID, and WiLI-2018.

B Dataset Provenance and Source Corpus Breakdown

Table 7 summarizes the provenance, orthographic representation, textual domains, sentence distributions, and repository links for all primary sources assembled into our 74,318-sentence target benchmark.

C Baseline Model Architectures and Hyperparameters

All seven from-scratch baseline models were trained using the following standardized configurations:

- **Multinomial Naive Bayes:** Scikit-learn implementation (Pedregosa et al., 2011); character n -gram range [1, 5]; sublinear term frequency scaling; Laplace smoothing parameter $\alpha = 0.1$.

- **Linear SVM:** Scikit-learn LinearSVC; character n -grams [1, 5]; sublinear TF-IDF; squared hinge loss; regularization parameter $C = 1.0$; tolerance 10^{-4} ; maximum iterations 2,000.

- **Logistic Regression:** Scikit-learn LogisticRegression; character n -grams [1, 5]; multinomial cross-entropy loss; L-BFGS solver; $C = 1.0$; maximum iterations 1,000.

- **XGBoost:** XGBClassifier (Chen and Guestrin, 2016); 100 estimators; learning rate $\eta = 0.1$; maximum tree depth 6; subsample ratio 0.8; colsample_bytree 0.8; multi:softprob objective.

- **Char-CNN:** PyTorch implementation (Zhang et al., 2015); character vocabulary size 200; character embedding dimension 64; 3 parallel 1D convolution banks with kernel sizes 3, 5, and 7 (128 filters each); ReLU activation; global max-pooling; dropout $p = 0.3$; AdamW optimizer (learning rate 10^{-3} , weight decay 10^{-4}); batch size 64; trained for 10 epochs with early stopping.

- **Char-BiGRU:** PyTorch implementation; character embedding dimension 64; 1-layer bidirectional GRU with 128 hidden units per direction; dropout $p = 0.3$; linear classification head ($256 \rightarrow 3$); AdamW optimizer (learning rate 10^{-3}); batch size 64; trained for 10 epochs with early stopping.

- **fastText (From Scratch):** Official C++ binary / Python API (Joulin et al., 2017); character subword n -gram range [3, 6]; embedding dimension 100; learning rate $\alpha = 0.5$; 10 epochs; hierarchical softmax loss; bucket size 2,000,000.

Table 4: FLORES+ Hybrid Benchmark Performance: Per-class F1 across all 11 target and global background languages.

Model	Sinh-Sinh	Pali-Sinh	San-Sinh	San-Deva	Eng-Latn	Tam-Taml	Hin-Deva	Ben-Beng	Ara-Arab	Fre-Latn	Ger-Latn	Macro-F1
<i>From-Scratch Baselines (110k Multi-Language Training)</i>												
MNB	0.9902	0.9781	0.9412	0.9850	0.9120	0.9820	0.9750	0.9650	0.9210	0.9450	0.9530	0.9589
Linear SVM	0.9920	0.9815	0.9540	0.9890	0.9350	0.9910	0.9820	0.9780	0.9520	0.9610	0.9670	0.9712
fastText	0.9915	0.9800	0.9490	0.9880	0.9240	0.9880	0.9790	0.9710	0.9410	0.9510	0.9590	0.9634
<i>Zero-Shot Foundation Models (Stock Base Weights)</i>												
fastText LID-176	0.6558	0.0000	0.0000	0.9868	0.9472	0.9946	0.9912	0.9926	0.9950	0.9792	1.0000	0.7021
OpenLID-v2	0.6542	0.0000	0.0000	0.9412	0.9280	0.9930	0.9850	0.9890	0.9850	0.9710	0.9980	0.6894
NLLB LID-218	0.6560	0.0000	0.0000	0.9870	0.9480	0.9940	0.9910	0.9930	0.9950	0.9800	1.0000	0.7042
GlotLID v3	0.6570	0.0000	0.0000	0.9890	0.9510	0.9950	0.9920	0.9940	0.9960	0.9820	1.0000	0.7088
ConLID	0.6580	0.0000	0.0000	0.9900	0.9530	0.9950	0.9930	0.9940	0.9960	0.9830	1.0000	0.7104
<i>Two-Stage Specialist Routing (Stock Foundation Router + Fine-Tuned Specialist)</i>												
fastText 2-Stage	0.9910	0.9785	0.9480	0.9868	0.9472	0.9946	0.9912	0.9926	0.9950	0.9792	1.0000	0.9721
OpenLID 2-Stage	0.9890	0.9740	0.9410	0.9412	0.9280	0.9930	0.9850	0.9890	0.9850	0.9710	0.9980	0.9685
NLLB 2-Stage	0.9920	0.9800	0.9510	0.9870	0.9480	0.9940	0.9910	0.9930	0.9950	0.9800	1.0000	0.9734
GlotLID 2-Stage	0.9925	0.9810	0.9520	0.9890	0.9510	0.9950	0.9920	0.9940	0.9960	0.9820	1.0000	0.9740
ConLID 2-Stage	0.9935	0.9825	0.9545	0.9900	0.9530	0.9950	0.9930	0.9940	0.9960	0.9830	1.0000	0.9752

Table 5: CommonLID Hybrid Benchmark Performance: Per-class F1 across all 11 languages.

Model	Sinh-Sinh	Pali-Sinh	San-Sinh	San-Deva	Eng-Latn	Tam-Taml	Hin-Deva	Ben-Beng	Ara-Arab	Fre-Latn	Ger-Latn	Macro-F1
fastText LID-176	0.6558	0.0000	0.0000	0.9862	0.9590	0.9818	0.9713	0.9931	0.9952	0.9471	0.9674	0.6980
OpenLID-v2	0.6542	0.0000	0.0000	0.0276	0.9427	0.9940	0.0134	0.9424	0.9810	0.9687	0.9817	0.6841
fastText 2-Stage	0.9910	0.9785	0.9480	0.9862	0.9590	0.9818	0.9713	0.9931	0.9952	0.9471	0.9674	0.9510
ConLID 2-Stage	0.9935	0.9825	0.9545	0.9880	0.9620	0.9850	0.9750	0.9940	0.9960	0.9510	0.9710	0.9561

Table 6: WiLI-2018 Hybrid Benchmark Performance: Per-class F1 across all 11 languages.

Model	Sinh-Sinh	Pali-Sinh	San-Sinh	San-Deva	Eng-Latn	Tam-Taml	Hin-Deva	Ben-Beng	Ara-Arab	Fre-Latn	Ger-Latn	Macro-F1
fastText LID-176	0.6558	0.0000	0.0000	0.9977	0.8887	0.9945	0.9894	0.9691	0.9710	0.9798	0.9703	0.7015
fastText 2-Stage	0.9910	0.9785	0.9480	0.9977	0.8887	0.9945	0.9894	0.9691	0.9710	0.9798	0.9703	0.9702
ConLID 2-Stage	0.9935	0.9825	0.9545	0.9980	0.8920	0.9950	0.9910	0.9710	0.9730	0.9820	0.9730	0.9738

Table 7: Target Corpus Provenance and Source Breakdown for Sinhala, Pali, and Sanskrit in Sinhala Script ($N = 74,318$). All source repositories are publicly accessible.

Language Class	Primary Source / Repository	Sentences	Orthography	Genre / Domain	Public Access Repository
Sinhala (Sinh-Sinh)	SiPaKosa-Sent	20,418	Native Sinhala	Buddhist Commentary & History	hf.co/datasets/RaniduG/SiPaKosa-Sent
	Pali-Sinhala Parallel	11,845	Native Sinhala	Canonical Translation Exegesis	hf.co/datasets/sinhala-nlp/pali-sinhala
Pali (Pali-Sinh)	SiDiaC-v.2.0	17,472	Native Sinhala	Canonical Tipitaka & Atthakatha	github.com/NeviduJ/SiDiaC-v.2.0
	Pali-Sinhala Parallel	11,845	Native Sinhala	Canonical Sutta Verses	hf.co/datasets/sinhala-nlp/pali-sinhala
Sanskrit (San-Sinh)	SansinNT	7,953	Native Sinhala	Sanskrit New Testament (27 Books)	ebible.org/details.php?id=sansin
	DCS (Kalidasa, Asvaghosa)	4,684	Transliterated	Classical Kavya & Shastra	github.com/ambuda-org/dcs
	SiDiaC (Kavacha Sangrahaya)	101	Native Sinhala	Protective Mantras & Stotras	github.com/NeviduJ/SiDiaC-v.2.0
Total Target Corpus		74,318			